

IncidentIQ

AI SRE Platform Video Script & Demo Guide

A comprehensive interactive walkthrough, case studies,
and full step-by-step production demo guide.

PROJECT METADATA:

Deployment: <https://incidentiq-1099197368634.us-central1.run.app>

GitHub Repo: <https://github.com/Hardik182005/IncidentIQ>

Author: Hardik Hinduja

IncidentIQ - 3-Minute Video Script + Demo Guide

QUICK ANSWERS TO YOUR QUESTIONS

Where is the Data Room?

The Data Room is inside the website at the sidebar navigation link called "Data Room".

- URL: ``https://incidentiq-1099197368634.us-central1.run.app/screens/integrations.html``
- It's the last item in the left sidebar of every page (Command Center, Incidents, Metrics, Alerts, Runbooks, Data Room)
- You do NOT need a separate URL - just click "Data Room" in the sidebar

Can you see logs in Grafana/New Relic/Datadog?

YES! The Data Room has "Open console " links for each provider:

- Datadog: Opens ``https://app.us5.datadoghq.com/logs?query=source:incidentiq-seed``
- Grafana Loki: Opens ``https://hardikhinduja399.grafana.net/explore``
- New Relic: Opens ``https://one.newrelic.com/logger``

After you Seed Pull, you can click those links to show the same logs inside each provider's actual UI.

What does "Push demo telemetry" on the landing page do?

The "Push demo telemetry" button on the landing page calls ``POST /api/integrations/seed``, which:

1. Pushes realistic error logs INTO Datadog (via Logs Intake API)
2. Pushes logs INTO Grafana Loki (via Loki push API)
3. Pushes logs INTO New Relic (via Log API)

This means real logs appear in their actual dashboards - proving IncidentIQ has live, bidirectional integration.

CHROME TABS TO OPEN BEFORE DEMO (7 tabs)

Open these tabs in this order before you start recording:

Tab #	URL	Purpose
-------	-----	---------

1	<code>`https://incidentiq-1099197368634.us-central1.run.app`</code>	**Landing page** - start here
2	<code>`https://incidentiq-1099197368634.us-central1.run.app/screens/dashboard.html`</code>	**Command Center** - main dashboard
3	<code>`https://incidentiq-1099197368634.us-central1.run.app/screens/integrations.html`</code>	**Data Room** - live proof of integrations
4	<code>`https://incidentiq-1099197368634.us-central1.run.app/screens/metrics.html`</code>	**Metrics** - service health
5	<code>`https://incidentiq-1099197368634.us-central1.run.app/screens/runbooks.html`</code>	**Runbooks** - response procedures
6	<code>`https://app.us5.datadoghq.com/logs?query=source:incidentiq-seed`</code>	**Datadog console** - see seeded logs
7	**Terminal / CMD** open	For running curl commands live

Optional bonus tabs: - ``https://hardikhinduja399.grafana.net/explore`` - Grafana Loki logs - ``https://one.newrelic.com/logger`` - New Relic logs - ``https://github.com/Hardik182005/IncidentIQ`` - GitHub repo

THE 3-MINUTE VIDEO SCRIPT

[0:00 - 0:25] OPENING - Problem + Hook

[Show: Landing page hero section with animated topology]

"During a production outage, every second matters. But here's what really happens - engineers spend the first 15 to 20 minutes just searching through logs, switching between Datadog, Grafana, New Relic dashboards, trying to figure out what went wrong.

>

What if an AI agent could do all of that in under 15 seconds?

>

This is IncidentIQ - an AI-powered Site Reliability Engineering platform that connects with your real monitoring tools, analyzes live telemetry, finds the root cause, and gives you the fix command. Let me show you."

[Action: Scroll down the landing page slowly to show the stats bar: 4.5min MTTR, 99.97% Uptime, 2,847 Incidents Resolved, 0.8s Detection Time]

[0:25 - 0:55] LIVE INTEGRATIONS - Proving it's real

[Action: Point to the "LIVE TELEMETRY INGEST" bar showing Datadog, Grafana Loki, New Relic]

"First, let me prove this is not mocked. You can see this bar here - Live Telemetry Ingest. We are connected to Datadog, Grafana Loki, and New Relic - all three - via their official APIs."

[Action: Click "Push Push demo telemetry" button on landing page]

"When I click 'Push demo telemetry', we're actually sending real error logs INTO these three tools through their APIs."

[Action: Switch to the Data Room tab (Tab 3)]

"Now let me show you the Data Room - this is our proof screen. You can see all three sources showing 'Connected'. Let me click 'Seed providers' to push logs, then 'Pull live data' to fetch them back."

[Action: Click "Push Seed providers" wait Click "Refresh Pull live data"]

"Look - we just pulled real records from all three tools. Each record is tagged with its source - Datadog, Grafana Loki, or New Relic. These are NOT fake - you can click 'Open Datadog console' right here and see the same logs inside Datadog itself."

[Action: Briefly switch to the Datadog tab (Tab 6) to show the seeded logs appear there too]

[0:55 - 1:45] CASE STUDY - Database Deadlock (The Money Demo)

[Action: Switch to Terminal tab (Tab 7)]

"Now let me show you the real power. I'm going to simulate a database deadlock - a real production scenario where two database transactions lock each other, causing everything to freeze."

[Action: Run this curl command in terminal:]

```
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/chaos -H "Content-Type: applica
```

"This just did three things: 1. Generated 1,500 realistic database logs showing the deadlock progression 2. Ran our three-model AI pipeline on those logs 3. Broadcast the result to every open dashboard"

>

Let's break down the pipeline. Stage 1 - Groq's Llama 3.3 70-billion parameter model does the initial triage - it identifies the anomaly type, severity, and affected services - in under 2 seconds.

>

Then Stage 2 - Google Gemini 2.0 Flash correlates the events - it finds the causal chain, the probable trigger, and the blast radius - which services were affected.

>

Finally Stage 3 - OpenAI GPT-4o-mini synthesizes the root cause, generates a real confidence score based on evidence, and produces copy-paste fix commands.

>

The confidence score is real - it's computed from the number of corroborating log lines, whether a trigger was found, and how specific the root cause is. Weak evidence gives 35%, strong evidence gives 95%+."

[Action: Switch to Command Center tab (Tab 2) - show the incident card that appeared via WebSocket]

"And look - the incident appeared here live, on the Command Center, via WebSocket - with the root cause, confidence score, affected services, and fix commands. Each fix command is risk-rated - LOW, MEDIUM, or HIGH - so you know what's safe to run."

[Action: Switch to Metrics tab (Tab 4)]

"Notice the Metrics screen - the postgres and payments services just degraded in real time because we have an active incident affecting them. This is live reactivity."

[1:45 - 2:15] MORE FEATURES - Voice, Runbooks, Slack

[Action: Switch back to Command Center (Tab 2), point to the Voice Orb]

"We also have a multilingual voice assistant powered by ElevenLabs. You can talk to ARIA in English, Hindi, Marathi, Tamil, Gujarati, and more - ask it 'What's the root cause?' and it responds with a spoken diagnosis grounded in the actual incident data."

[Action: Switch to Runbooks tab (Tab 5)]

"The Runbooks page has pre-built incident response procedures - like 'DB Connection Pool Exhaustion' - with step-by-step commands. You can also create your own runbooks, and they're persisted via a real API. When a Slack alert fires, the 'Open Runbook' button deep-links straight to the matching runbook."

[Action: Show creating a new runbook - click "+ New Runbook", fill in Title, steps, and save]

"And the Slack integration sends rich Block Kit alert cards with buttons - 'View Dashboard' opens the exact incident, 'Open Runbook' opens the matching procedure. We have a live Slack workspace you can join to watch alerts land in real time."

[2:15 - 2:45] ARCHITECTURE - Under the hood

[Action: Switch to GitHub tab or show the architecture diagram from README]

"Under the hood, it's one Docker container deployed on Google Cloud Run. The backend is FastAPI with Python - all routes, WebSockets, and the 3-stage pipeline run here."

>

We have 10 backend services - one for each AI provider, one for each monitoring tool, plus a chaos engine and monitoring agent. The frontend is React 18 with Babel standalone - no build step needed.

>

All three monitoring connectors are bidirectional - we can push demo data INTO them and pull real records back. The Data Room is the proof of that.

>

To verify everything is live, you can just hit the health endpoint:"

[Action: In terminal, run:]

```
curl https://incidentiq-1099197368634.us-central1.run.app/api/health
```

"All four AI providers - Groq, Gemini, OpenAI, ElevenLabs - are green."

```
curl https://incidentiq-1099197368634.us-central1.run.app/api/integrations/status
```

"And all three monitoring tools - Datadog, Grafana, New Relic - are connected and authenticated."

[2:45 - 3:00] CLOSING - Call to Action

[Action: Switch back to Landing page (Tab 1)]

"IncidentIQ turns the slowest, most stressful part of incident response into a 15-second automated flow. Three AI models, three real monitoring tools, one unified platform.

>

The live app is deployed on Google Cloud Run - the link is in the description. You can trigger a chaos scenario right now and watch the AI diagnose it end-to-end.

>

Built to give engineers their incident-response time back. Thank you!"

CASE STUDY: "The Database Deadlock at 2 AM"

The Story (what you narrate)

"Imagine it's 2 AM. Your payments service suddenly starts throwing 500 errors. Orders are failing. Revenue is dropping every second. Your on-call engineer wakes up, opens Datadog - sees errors. Switches to Grafana - more errors. Goes to New Relic - connection failures.

>

20 minutes later, they finally find it: a database deadlock between the orders table and the transactions table. Two processes locked each other. Everything froze.

>

With IncidentIQ, here's what happens instead: The AI detects the anomaly in under 1 second. Groq triages it - 'deadlock detected, critical severity'. Gemini correlates the timeline - finds the exact two processes that caused the lock. OpenAI gives the root cause, the fix command, and a prevention plan. Total time: 12 seconds.

>

That's the difference between a 20-minute scramble and a 12-second diagnosis."

The Commands (run these during the demo)


```
# Step 1: Check AI providers are alive
curl https://incidentiq-1099197368634.us-central1.run.app/api/health

# Step 2: Check monitoring tool connections
curl https://incidentiq-1099197368634.us-central1.run.app/api/integrations/status

# Step 3: Trigger the deadlock scenario
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/chaos \
  -H "Content-Type: application/json" \
  -d '{"scenario":"database_deadlock"}'

# Step 4: Push demo telemetry into providers
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/integrations/seed \
  -H "Content-Type: application/json" -d "{}"

# Step 5: Pull real data back from providers
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/integrations/sync \
  -H "Content-Type: application/json" \
  -d '{"window_minutes":120,"auto_analyze":false}'
```

What the AI response looks like (example)

The chaos endpoint returns a full JSON with:

- ``severity``: "critical"
- ``affected_services``: ["postgres", "payments-api", "order-service"]
- ``root_cause``: "Database deadlock between concurrent transactions on the orders and transactions tables"
- ``confidence``: 0.87 (computed from evidence, not hardcoded)
- ``fix_commands``: Risk-rated kubectl/psql commands
- ``voice_summary``: Spoken briefing for the voice assistant
- ``stage1_groq_ms``: ~800ms
- ``stage2_gemini_ms``: ~2100ms
- ``stage3_openai_ms``: ~4200ms
- ``total_processing_ms``: ~7200ms

STEP-BY-STEP DEMO WALKTHROUGH

Phase 1: Landing Page (30 seconds)

1. Open ``https://incidentiq-1099197368634.us-central1.run.app``
2. Point out the animated service topology on the right (canvas animation showing nodes like Datadog,

- Grafana, New Relic, gateway, payments, postgres, redis - with flowing data packets)
- 3. Point out the scrolling ticker at bottom showing service health status
- 4. Point out the "LIVE TELEMETRY INGEST" bar showing Datadog, Grafana Loki, New Relic with green dots
- 5. Click "Push Push demo telemetry" - it shows "[OK] Pushed datadog, grafana, newrelic"
- 6. Scroll down to show stats (4.5 min MTTR, 99.97% uptime, etc.)
- 7. Scroll to Features section - 6 features: Multi-Model AI, Real-time Observability, Voice Interface, Chaos Engineering, AI Fix Commands, Incident Timeline
- 8. Scroll to "How it works" - 3 steps: Detect Diagnose Resolve
- 9. Scroll to Terminal animation - auto-plays showing the AI pipeline analyzing an incident

Phase 2: Data Room (30 seconds)

- 1. Navigate to Data Room (click sidebar or go to Tab 3)
- 2. Show the 3 source cards: Datadog (* CONNECTED), Grafana Loki (* CONNECTED), New Relic (* CONNECTED)
- 3. Click "Push Seed providers" toast says "Pushed to Datadog, Grafana Loki, New Relic"
- 4. Click "Refresh Pull live data" watch records stream in
- 5. Point out each record is color-tagged: Purple = Datadog, Orange = Grafana Loki, Green = New Relic
- 6. Point out the topbar: "SOURCES LIVE: 3/3", "RECORDS PULLED: 96+"
- 7. Click "Open Datadog console " to show logs in actual Datadog

Phase 3: Chaos Scenario via Terminal (45 seconds)

- 1. Switch to Terminal
- 2. Run the ``database_deadlock`` chaos command
- 3. Switch to Command Center - the incident appeared live via WebSocket
- 4. Expand the incident card - show root cause, confidence, fix commands
- 5. Switch to Metrics - show postgres and payments degraded
- 6. Switch to Incidents tab - show incident in the table

Phase 4: Voice + Runbooks + Slack (30 seconds)

- 1. On Command Center, show the Voice Orb (bottom right) - mention multilingual support
- 2. Switch to Runbooks - show the pre-built "DB Connection Pool Exhaustion" runbook
- 3. Click "View Steps" to expand - show the kubectl/psql commands
- 4. Click "+ New Runbook" to show the creation modal
- 5. Mention Slack integration - alerts land with deep-link buttons

Phase 5: Alerts Page (15 seconds)

1. Switch to Alerts page
 2. Show 3 active alerts firing
 3. Show the Alert Rules table (12 rules with toggles)
 4. Show the Notification Channels (Slack, PagerDuty, Email, Webhook)
 5. Point out the "Join Slack to see alerts " button
-

KEY TALKING POINTS (for any Q&A)

Question	Answer
"Is the data real?"	Yes - we push into Datadog/Grafana/New Relic via their APIs, and pull it back. The Data Room proves it.
"Is the confidence score hardcoded?"	No - it's computed from evidence: number of corroborating log lines, whether correlation found a trigger + blast radius, root-cause specificity. It genuinely varies per incident.
"How many AI models?"	3: Groq Llama 3.3 70B (triage), Gemini 2.0 Flash (correlation), OpenAI GPT-4o-mini (root cause). They run sequentially - each feeds the next.
"What if one AI model is down?"	The pipeline is resilient - each stage catches its own errors. If one fails, the others still return a partial diagnosis and confidence drops honestly.
"What chaos scenarios?"	5: `database_deadlock`, `db_connection_leak`, `memory_leak`, `network_latency_spike`, `api_cascade`
"What languages does the voice assistant support?"	English, Hindi, Marathi, Tamil, Telugu, Bengali, Gujarati, Punjabi, Kannada, Malayalam
"Where is it deployed?"	Google Cloud Run, single container with FastAPI + static frontend

CASE STUDY 2: "The Reliance Jio Outage (API Cascade Failure)"

The Story (what you narrate)

"Imagine an outage of national scale, similar to a Reliance Jio Fiber core switch degradation or gateway timeout cascade. Millions of users are experiencing payment errors or connectivity drops. The payment gateway is timing out under a massive retry storm.

>

In a traditional setup, SREs are running traceroutes, digging through massive router dumps, checking pgbouncer logs, trying to isolate if it's a network loop, a core database deadlock, or an external vendor outage. MTTR is measured in hours.

>

With IncidentIQ, here's what happens instead: The simulated network latency spike or API cascade is triggered. Groq detects the latency issue in under 1 second. Gemini maps the blast radius across payment-service and api-gateway. OpenAI GPT-4o-mini generates the copy-paste fix commands, isolates the eth0 buffer overrun on the switch, and gives the SRE a safe rollback play.

>

Total time: 14 seconds. That's the difference between national-scale network panic and a 14-second diagnosis."

The Commands (run these during the demo)

```
# Option A: Trigger the Network Latency Spike (Simulating router degradation)
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/chaos \
  -H "Content-Type: application/json" \
  -d '{"scenario":"network_latency_spike"}'

# Option B: Trigger the API Cascade Failure (Simulating webhook timeout cascade)
curl -X POST https://incidentiq-1099197368634.us-central1.run.app/api/chaos \
  -H "Content-Type: application/json" \
  -d '{"scenario":"api_cascade"}'
```

What the AI Response looks like (example)

The chaos endpoint returns a full JSON containing:

- ``severity``: "critical"
- ``affected_services``: ["api-gateway", "payments-api", "network-switch"]
- ``root_cause``: "Transit latency spike due to buffer overrun on eth0 interface of network-switch router"
- ``confidence``: 0.94 (calculated from active alert correlations)

- ``fix_commands``: ["kubectl set env deployment/api-gateway RETRY_MAX_LIMIT=3;", "kubectl rollout restart deployment/network-switch;"]
- ``voice_summary``: "A major transit latency spike has been detected on the gateway switch, causing payments-api gateway timeouts. We recommend resetting the switch configuration."

ALL URLS

Resource	URL
Live App	https://incidentiq-1099197368634.us-central1.run.app
GitHub	https://github.com/Hardik182005/IncidentIQ
Command Center	https://incidentiq-1099197368634.us-central1.run.app/screens/dashboard.html
Incidents	https://incidentiq-1099197368634.us-central1.run.app/screens/incidents.html
Metrics	https://incidentiq-1099197368634.us-central1.run.app/screens/metrics.html
Alerts	https://incidentiq-1099197368634.us-central1.run.app/screens/alerts.html
Runbooks	https://incidentiq-1099197368634.us-central1.run.app/screens/runbooks.html
Data Room	https://incidentiq-1099197368634.us-central1.run.app/screens/integrations.html
Health API	https://incidentiq-1099197368634.us-central1.run.app/api/health
Integration Status	https://incidentiq-1099197368634.us-central1.run.app/api/integrations/status
Slack	https://join.slack.com/t/incidentiq-world/shared_invite/zt-3yu7eu01h-cov54rryirD67XPYz97eOw